



12.3. Heap Sort

"Build the Heap. Extract the Greatest. Repeat Until Sorted."

Previous Section:

 [12.2. Operations on Heaps](#)

Contents:

[1. What is Heap Sort?](#)

[2. Demonstration of Heap Sort](#)

[3. Implementation of Heap Sort](#)

[Summary](#)

[References](#)

Up until now, we have learned that heaps are excellent at maintaining the largest (or smallest) element at the root of the tree. Because of this unique property, one of the most famous applications of heaps is **Heap Sort**.

Unlike many sorting algorithms that repeatedly compare neighboring elements, Heap Sort takes advantage of the Binary Heap structure to efficiently locate and remove the largest element one by one. By repeatedly extracting the root node and placing it into its correct position, the entire array eventually becomes sorted.

Heap Sort is particularly attractive because it guarantees a time complexity of **$O(n \log n)$** regardless of the input arrangement, making it one of the most reliable comparison-based sorting algorithms.

1. What is Heap Sort?

Imagine that you have a pile of exam papers and need to arrange them from the lowest score to the highest score. One way would be to repeatedly search through the entire pile to find the highest score, place it at the end, and then repeat the process for the remaining papers.

A Max Heap performs this idea much more efficiently. Heap Sort is a sorting algorithm based on the **Binary Heap** data structure. It works by repeatedly removing the largest element from a Max Heap and placing it into its final position in the array. The process continues until no elements remain in the heap.

The Core Idea - A Max Heap always guarantees that the largest element is stored at the root node. Therefore:

1. Access the root (largest value).
2. Move it to the end of the array.
3. Remove it from the heap.
4. Restore the heap property using Heapify.
5. Repeat until all elements are sorted.

Eventually, the array becomes arranged in ascending order.

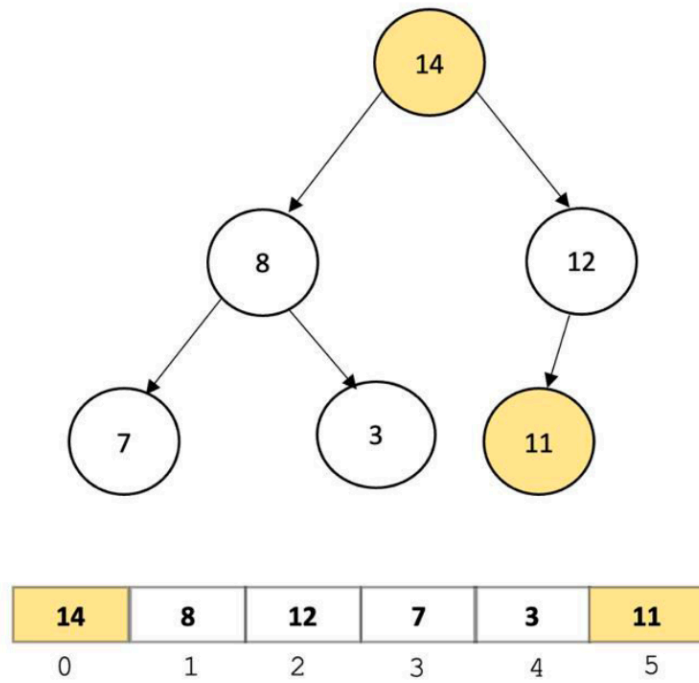
Applications of Heap Sort

Heap Sort is commonly used when:

- Guaranteed $O(n \log n)$ performance is required.
- Memory usage must remain low.
- A stable running time is more important than average-case speed.
- Priority Queue data needs to be converted into a sorted sequence.

2. Demonstration of Heap Sort

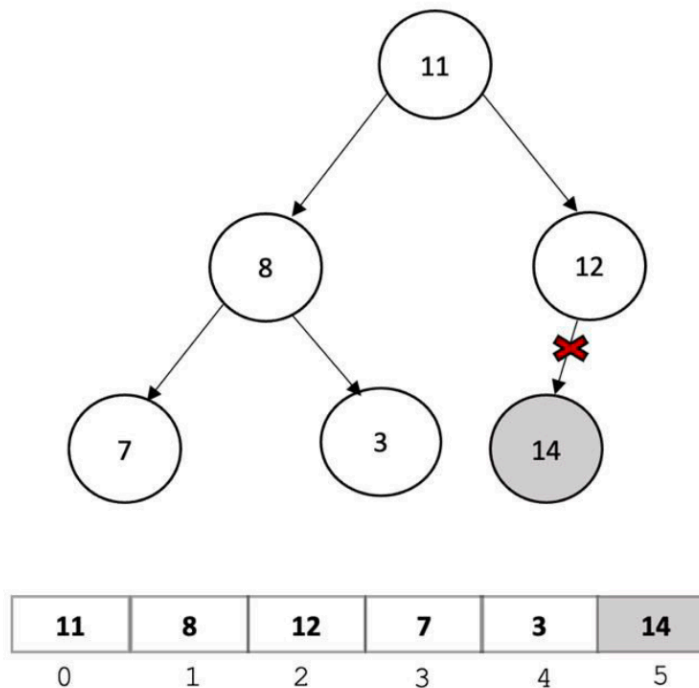
Let us visualize the entire process using the following array: `[14, 8, 12, 3, 7, 11]` .
After building a Max Heap:



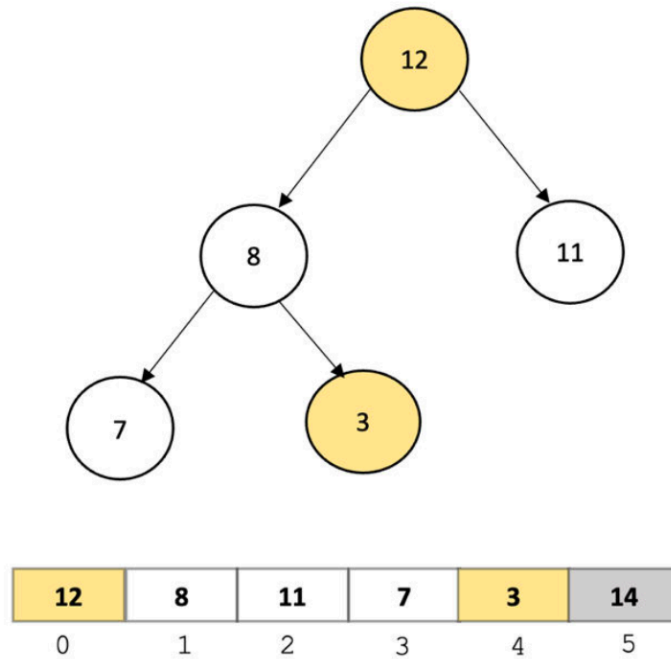
Notice that the largest value (**14**) is already located at the root.

- **Step 1 – Extract 14**

Move 14 to the last position. Replace the root with the last node 11.

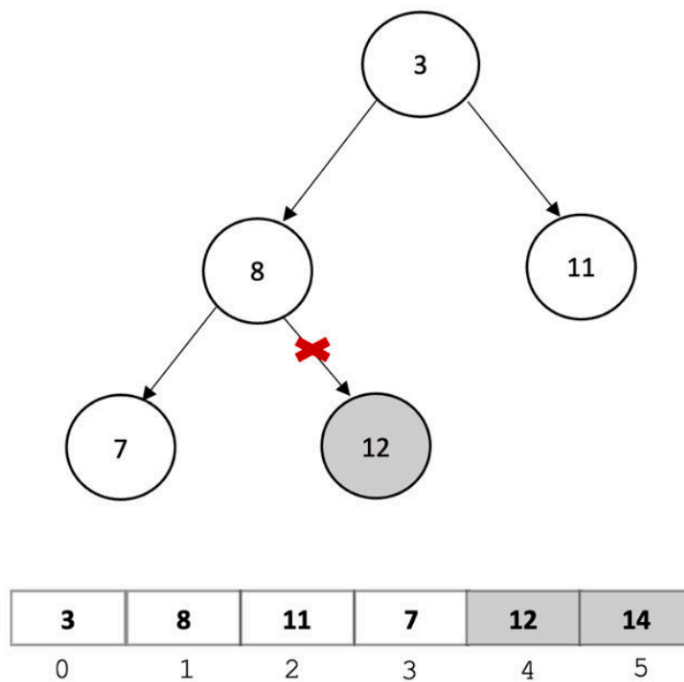


Heap property is violated because: $11 < 12$. Heapify:

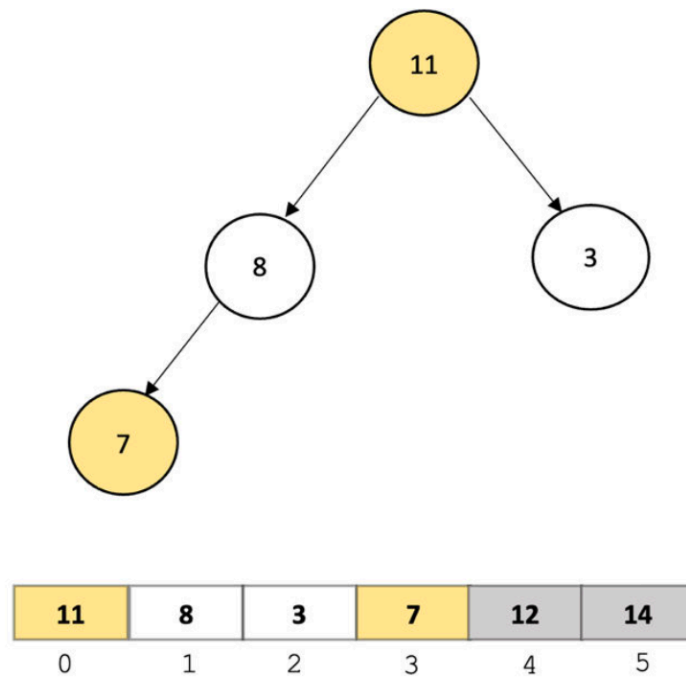


- **Step 2 – Extract 12**

Move 12 to the second-last position. Replace root with 7.

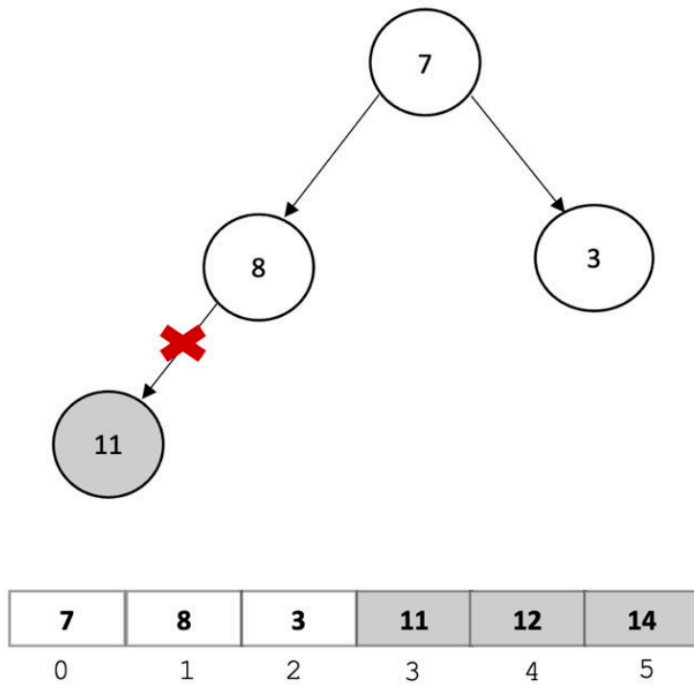


Heapify:

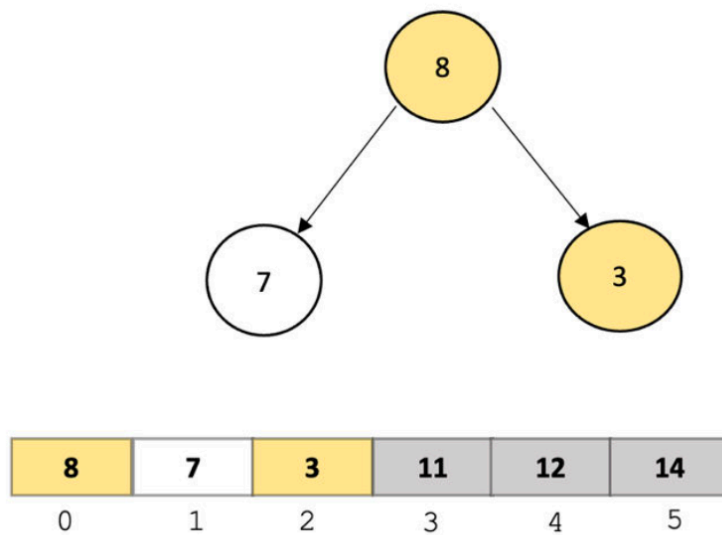


- **Step 3 – Extract 11**

Move 11 to its sorted position. Replace root with 3.

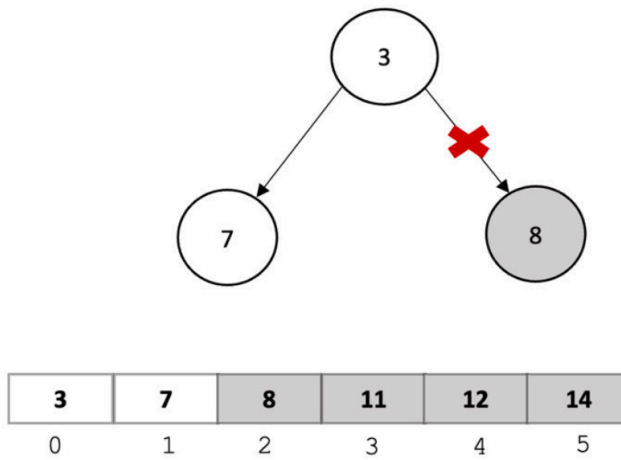


Heapify:

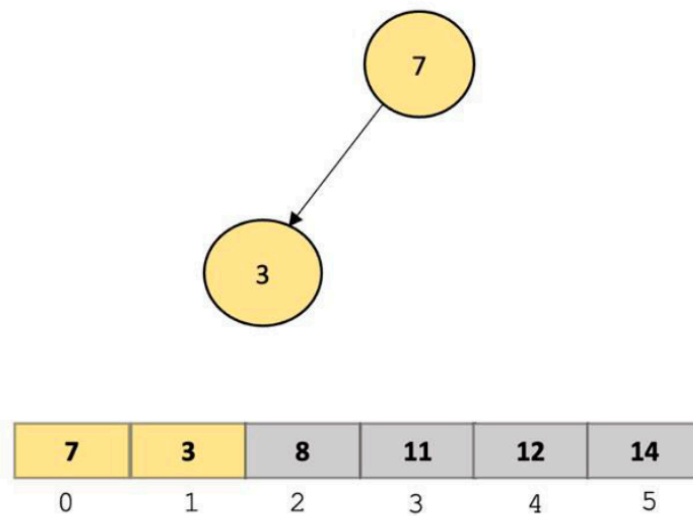


- **Step 4 – Extract 8**

Move 8 to its sorted position. Replace root with 7.

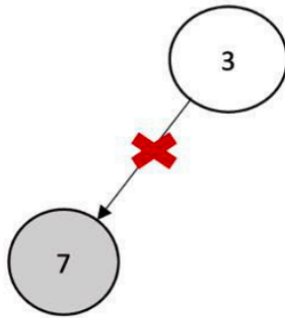


Heap property already holds.



- **Step 5 – Extract 7**

Move 7 to its sorted position. Replace root with 3.



3	7	8	11	12	14
0	1	2	3	4	5

• Step 6 – Extract 3

Move the final element.



3	7	8	11	12	14
0	1	2	3	4	5

3	7	8	11	12	14
0	1	2	3	4	5

Heap is now empty. The array has been successfully sorted in ascending order.

Sorted Portion: [3, 7, 8, 11, 12, 14]

Notice what happens during every iteration:

- The largest element is always removed first.
- Heapify restores the heap structure.
- The sorted portion grows from right to left.
- The heap portion becomes smaller and smaller.

3. Implementation of Heap Sort

Use this code for the heap class

`heap.py`

Now let us implement Heap Sort using the Max Heap class that we developed previously.

```
def heap_sort(array):
    # Create a Max Heap
    heap = MaxHeap(len(array))

    # Insert all elements into heap
    for num in array:
        heap.insert(num)

    sorted_array = []

    # Continue until heap becomes empty
    while heap.last >= 0:

        # Retrieve the maximum value
        max_value = heap.heap[heap.peak]

        # Store the extracted value
        sorted_array.append(max_value)

        # Remove the root node
        heap.delete()

        # Heapify is automatically performed
        # inside the delete operation

    # Reverse because values were extracted
```

```
# from largest to smallest
return sorted_array[::-1]
```

```
numbers = [14, 8, 12, 3, 7, 11]

result = heap_sort(numbers)
print(result)
```

[3, 7, 8, 11, 12, 14]

Time Complexity Analysis - Heap Sort consists of two major phases:

- Building the Heap: Inserting all elements into the heap, with n insertions and each insertion = $O(\log n) \rightarrow Total = O(n \log n)$
- Repeated Deletion: Removing all elements, with n deletions and each deletion = $O(\log n) \rightarrow Total = O(n \log n)$
- Space Complexity:
 - If a separate heap and output array are used: $O(n)$
 - If Heap Sort is performed directly on the array itself: $O(1)$ with additional memory.

Heap Sort Visualization

<https://www.cs.usfca.edu/~galles/visualization/HeapSort.html>

Summary

Heap Sort is a comparison-based sorting algorithm that utilizes the Binary Heap structure to efficiently sort data.

The algorithm repeatedly removes the largest element from a Max Heap, places it into its final sorted position, and then restores the heap property through Heapify. Because the root node always contains the maximum value, Heap Sort can continuously extract elements in descending order and build an ascending sorted array.

One of Heap Sort's greatest strengths is its guaranteed **$O(n \log n)$** running time regardless of the input arrangement. While other sorting algorithms may experience poor performance in certain cases, Heap Sort remains consistently efficient by relying on the structural properties of heaps.

By combining the power of **Heap Construction**, **Deletion**, and **Heapify**, Heap Sort transforms a heap into a fully sorted sequence, making it one of the most important practical applications of the Binary Heap data structure.

References

https://drive.google.com/file/d/1VfNgqFPkvnDGZeVkN_QqIHjWCDQRapBM/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/heap-sort/>

<https://www.programiz.com/dsa/heap-sort>

<https://www.youtube.com/watch?v=HqPJF2L5h9U>

Next Section:

 [12.4. Huffman Coding](#)